

Task No.	Task 2
Title	Student Profile Manager using JavaScript Class and DOM
Course Outcome	CO1

1. Aim

To create a Student Profile Manager web application using a JavaScript class and DOM manipulation, which accepts student details through a form and dynamically generates a styled profile card on the webpage.

2. Objective

To understand and apply JavaScript classes and objects, DOM element selection, dynamic creation of HTML elements, event handling, and basic CSS styling of dynamically generated content.

3. Theory

A JavaScript class is a template for creating objects that bundle related data (properties) together — here, a Student class holds name, roll number, department, and CGPA. An object of this class is created using the new keyword with values read from input fields.

The Document Object Model (DOM) represents the webpage as a tree of elements that JavaScript can read and modify. `document.getElementById()` selects existing elements (form inputs, the container div), while `document.createElement()` dynamically builds new elements at runtime.

Event handling, via `addEventListener()`, allows the page to respond to user actions — here, a click on the “Generate Profile” button triggers a function that reads the input values, validates them, creates a Student object, builds a new profile card using DOM methods (`createElement`, `textContent`, `appendChild`), and inserts it into the page. CSS classes and a keyframe animation give the generated card its styled, fade-in appearance.

4. Algorithm

1. Start.
2. Create a valid HTML5 document and set the page `<title>` to “Student Profile Manager”.
3. Design a form with input fields for Name, Roll Number, Department, and CGPA, and a “Generate Profile” button.
4. Define a JavaScript Student class with a constructor that stores name, roll, dept, and cgpa as properties.
5. Select the button and the empty profile container using `getElementById()`.
6. Add a click event listener to the “Generate Profile” button.
7. Inside the event handler, read and trim the values of all four input fields.
8. Validate that no field is empty; if any field is empty, show an alert and stop.
9. Create a new Student object using the entered values.
10. Clear any previous profile card from the container.
11. Dynamically create a card `<div>` and heading/paragraph elements using `createElement()`, and set their text using `textContent`.
12. Append the new elements to the card, and append the card to the profile container using `appendChild()`.
13. Save the file, open it in a browser, fill the form, click Generate Profile, and verify the profile card appears correctly.
14. Stop.

5. Program (Source Code) — task2.html

task2.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Student Profile Manager</title>
<style>
  body {
    font-family: 'Segoe UI', Arial, sans-serif;
    background: #f0f4f8;
    display: flex;
    flex-direction: column;
    align-items: center;
    padding: 40px;
  }
  .form-container {
    background: #fff;
    padding: 25px 30px;
    border-radius: 10px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    width: 320px;
  }
  .form-container h2 {
    text-align: center;
    color: #2c3e50;
  }
  .form-container input {
    width: 100%;
    padding: 8px;
    margin: 8px 0;
    border: 1px solid #ccc;
    border-radius: 5px;
    box-sizing: border-box;
  }
  .form-container button {
    width: 100%;
    padding: 10px;
    margin-top: 10px;
    background: #2c7be5;
    color: #fff;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    font-size: 15px;
  }
}
```

```
.form-container button:hover {
  background: #1a5cbf;
}

.profile-card {
  margin-top: 25px;
  background: #ffffff;
  border-left: 6px solid #2c7be5;
  border-radius: 8px;
  padding: 20px 25px;
  width: 300px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
  animation: fadeIn 0.4s ease-in-out;
}

.profile-card h3 {
  margin-top: 0;
  color: #2c7be5;
}

.profile-card p {
  margin: 6px 0;
  color: #333;
}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(-10px); }
  to { opacity: 1; transform: translateY(0); }
}

</style>
</head>
<body>

<div class="form-container">
  <h2>Student Profile Manager</h2>
  <input type="text" id="name" placeholder="Enter Name">
  <input type="text" id="roll" placeholder="Enter Roll Number">
  <input type="text" id="dept" placeholder="Enter Department">
  <input type="text" id="cgpa" placeholder="Enter CGPA">
  <button id="generateBtn">Generate Profile</button>
</div>

<div id="profileContainer"></div>

<script>
  class Student {
    constructor(name, roll, dept, cgpa) {
      this.name = name;
      this.roll = roll;
```

```
this.dept = dept;
this.cgpa = cgpa;
}
}

const generateBtn = document.getElementById('generateBtn');
const profileContainer = document.getElementById('profileContainer');

generateBtn.addEventListener('click', function () {
  const nameVal = document.getElementById('name').value.trim();
  const rollVal = document.getElementById('roll').value.trim();
  const deptVal = document.getElementById('dept').value.trim();
  const cgpaVal = document.getElementById('cgpa').value.trim();

  if (!nameVal || !rollVal || !deptVal || !cgpaVal) {
    alert('Please fill all fields before generating the profile.');
```

return;

```
  }

  const student = new Student(nameVal, rollVal, deptVal, cgpaVal);

  profileContainer.innerHTML = '';

  const card = document.createElement('div');
  card.classList.add('profile-card');

  const heading = document.createElement('h3');
  heading.textContent = 'Student Profile';

  const nameP = document.createElement('p');
  nameP.textContent = `Name: ${student.name}`;

  const rollP = document.createElement('p');
  rollP.textContent = `Roll Number: ${student.roll}`;

  const deptP = document.createElement('p');
  deptP.textContent = `Department: ${student.dept}`;

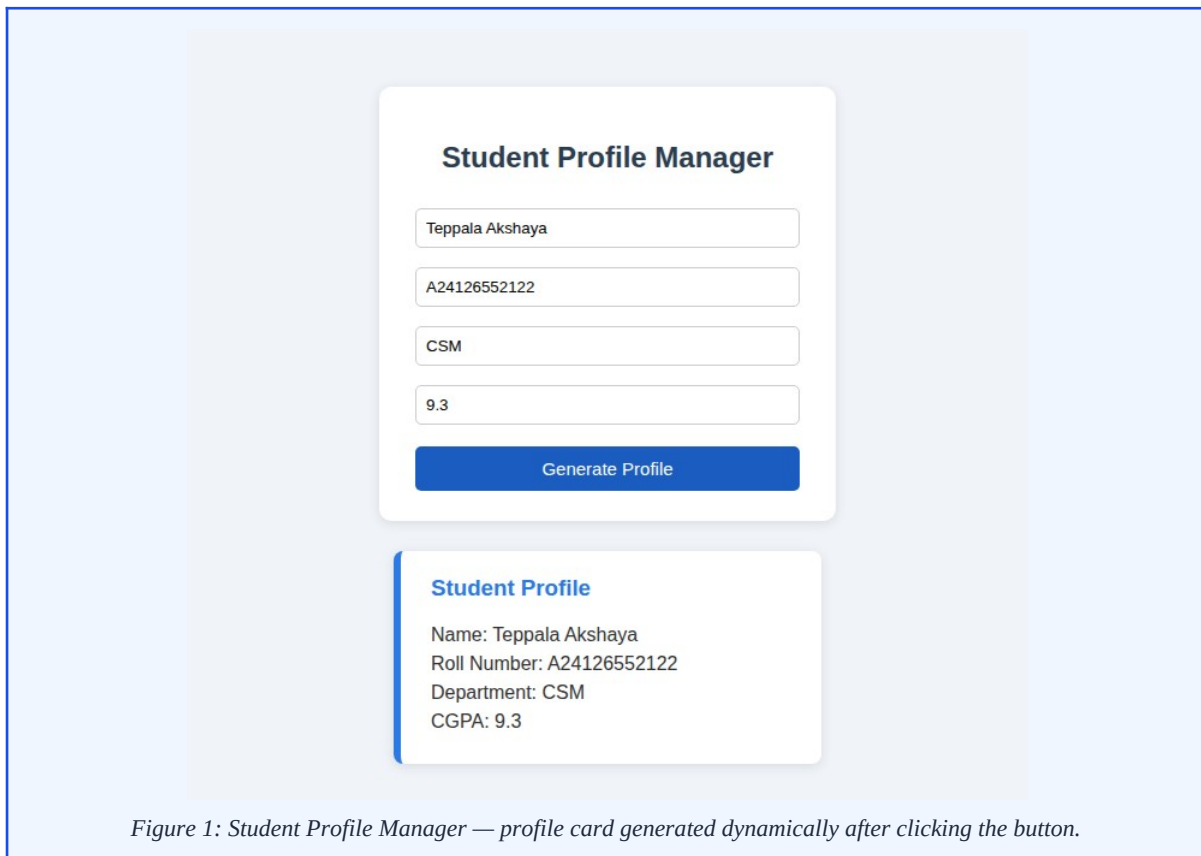
  const cgpaP = document.createElement('p');
  cgpaP.textContent = `CGPA: ${student.cgpa}`;

  card.appendChild(heading);
  card.appendChild(nameP);
  card.appendChild(rollP);
  card.appendChild(deptP);
```

```
card.appendChild(cgpaP);  
  
    profileContainer.appendChild(card);  
});  
</script>  
  
</body>  
</html>
```

6. Expected Output

The screenshot below shows task2.html rendered in a Chromium browser after filling in the form (Name, Roll Number, Department, CGPA) and clicking “Generate Profile”, showing the profile card dynamically created and inserted into the page via DOM manipulation.



7. Result

The Student Profile Manager was successfully created and executed using a JavaScript class and DOM manipulation. On clicking “Generate Profile”, a Student object was created from the entered values, and a styled profile card was dynamically built and inserted into the page, correctly demonstrating classes and objects, DOM selection, dynamic element creation, content updates, and event handling, fulfilling Course Outcome CO1.

Student Signature

Faculty / Lab In-charge Signature